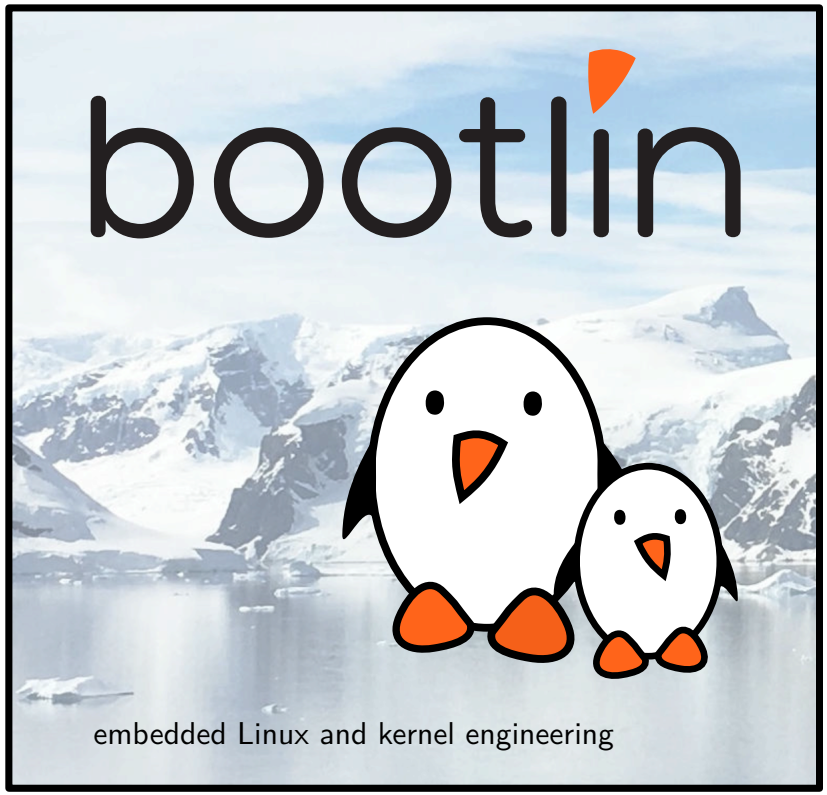
Cross-compiling toolchains



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



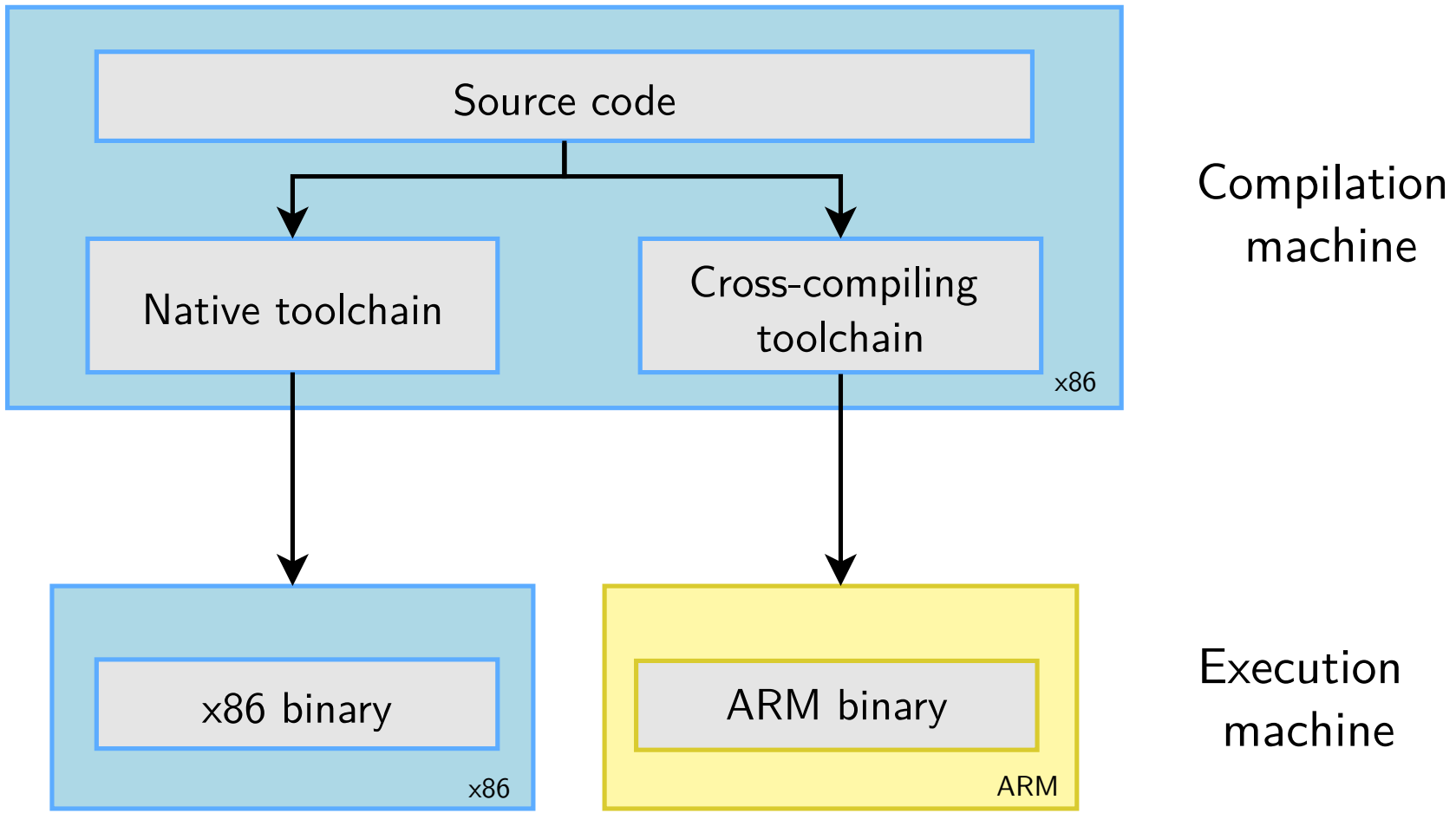
Definition and Components



Toolchain definition (1)

- The usual development tools available on a GNU/Linux workstation is a **native toolchain**
- This toolchain runs on your workstation and generates binary code for your workstation, usually x86
- For embedded system development, it is usually impossible or not interesting to use a native toolchain
  - The target is too restricted in terms of storage and/or memory
  - The target is very slow compared to your workstation
  - You may not want to install all development tools on your target.
- Therefore, **cross-compiling toolchains** are generally used. They run on your workstation but generate code for your target.

Toolchain definition (2)



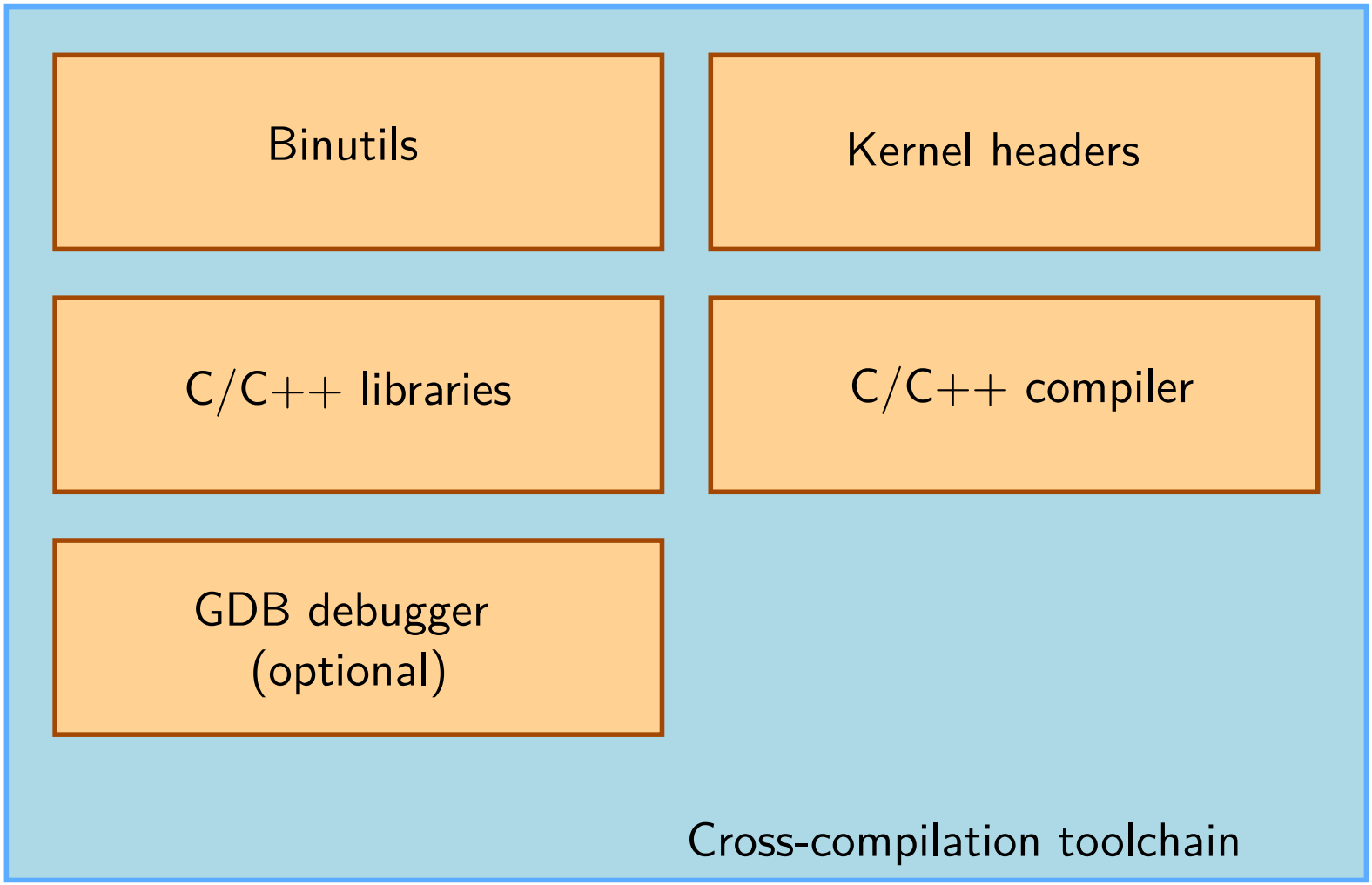


Architecture tuple and toolchain prefix

- Many UNIX/Linux build mechanisms rely on *architecture tuple* names to identify machines.
- Examples: `arm-linux-gnueabihf`, `mips64el-linux-gnu`, `arm-vendor-none-eabihf`
- These tuples are 3 or 4 parts:
  1. The architecture name: `arm`, `riscv`, `mips64el`, etc.
  2. Optionally, a vendor name, which is a free-form string
  3. An operating system name, or `none` when not targeting an operating system
  4. The ABI/C library (see later)
- This tuple is used to:
  - configure/build software for a given platform
  - as a prefix of cross-compilation tools, to differentiate them from the native toolchain
    - `gcc` → native compiler
    - `arm-linux-gnueabihf-gcc` → cross-compiler



Components of gcc toolchains





Binutils

- **Binutils** is a set of tools to generate and manipulate binaries (usually with the ELF format) for a given CPU architecture
  - `as`, the assembler, that generates binary code from assembler source code
  - `ld`, the linker
  - `ar`, `ranlib`, to generate `.a` archives (static libraries)
  - `objdump`, `readelf`, `size`, `nm`, `strings`, to inspect binaries. Very useful analysis tools!
  - `objcopy`, to modify binaries
  - `strip`, to strip parts of binaries that are just needed for debugging (reducing their size).
- GNU Binutils: https://www.gnu.org/software/binutils/, GPL license



C/C++ compiler

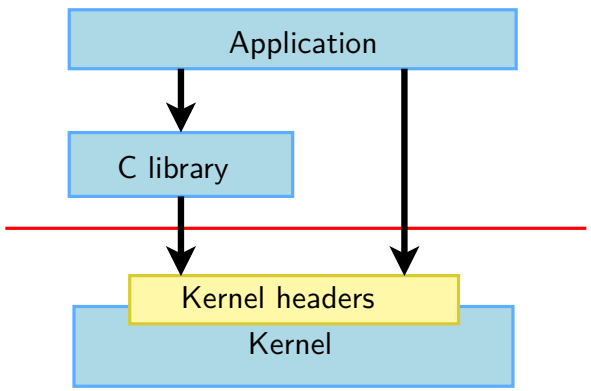
- GCC: GNU Compiler Collection, the famous free software compiler
- https://gcc.gnu.org/
- Can compile C, C++, Ada, Fortran, Java, Objective-C, Objective-C++, Go, etc. Can generate code for a large number of CPU architectures, including x86, ARM, RISC-V, and many others.
- Available under the GPL license, libraries under the GPL with linking exception.





Kernel headers (1)

- The C standard library and compiled programs need to interact with the kernel
  - Available system calls and their numbers
  - Constant definitions
  - Data structures, etc.
- Therefore, compiling the C standard library requires kernel headers, and many applications also require them.
- Available in `<linux/...>` and `<asm/...>` and a few other directories corresponding to the ones visible in include/uapi/ and in `arch/<arch>/include/uapi` in the kernel sources
- The kernel headers are extracted from the kernel sources using the `headers_install` kernel Makefile target.





Kernel headers (2)

- System call numbers, in `<asm/unistd.h>`

```
#define __NR_exit          1
#define __NR_fork          2
#define __NR_read          3
```

- Constant definitions, here in `<asm-generic/fcntl.h>`, included from `<asm/fcntl.h>`, included from `<linux/fcntl.h>`

```
#define O_RDWR 00000002
```

- Data structures, here in `<asm/stat.h>` (used by the `stat` command)

```
struct stat {
    unsigned long st_dev;
    unsigned long st_ino;
    [...]
};
```



Kernel headers (3)

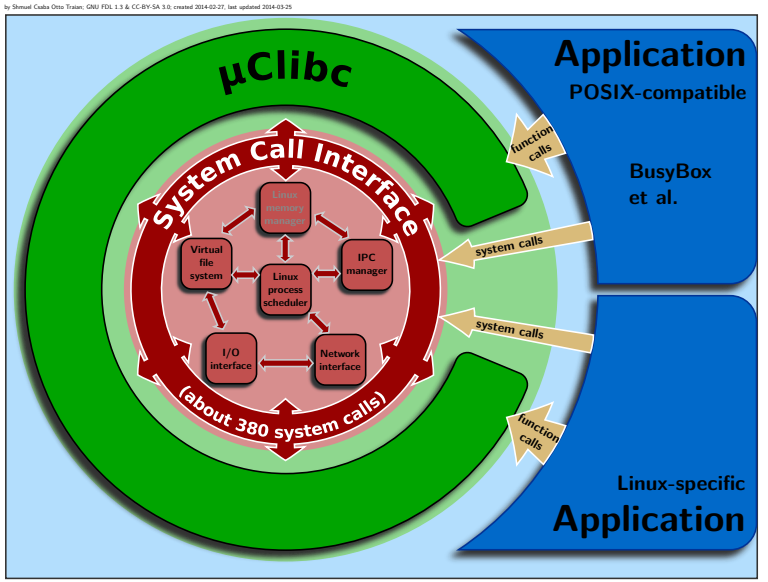
The kernel to user space interface is **backward compatible**

- Kernel developers are doing their best to **never** break existing programs when the kernel is upgraded. Otherwise, users would stick to older kernels, which would be bad for everyone.
- Hence, binaries generated with a toolchain using kernel headers older than the running kernel will work without problem, but won't be able to use the new system calls, data structures, etc.
- Binaries generated with a toolchain using kernel headers newer than the running kernel might work only if they don't use the recent features, otherwise they will break.

What to remember: updating your kernel shouldn't break your programs; it's usually fine to keep an old toolchain as long as it works fine for your project.

C standard library

- The C standard library is an essential component of a Linux system.
  - Interface between the applications and the kernel
  - Provides the well-known standard C API to ease application development
- Several C standard libraries are available: *glibc*, *uClibc*, *musl*, *klibc*, *newlib*...
- The choice of the C standard library must be made at cross-compiling toolchain generation time, as the GCC compiler is compiled against a specific C standard library.



Source: Wikipedia (https://bit.ly/2zrGve2)



glibc

- License: LGPL
- C standard library from the GNU project
- Designed for performance, standards compliance and portability
- Found on all GNU / Linux host systems
- Of course, actively maintained
- By default, quite big for small embedded systems. On armv7hf, version 2.31: `libc`: 1.5 MB, `libm`: 432 KB, source: https://toolchains.bootlin.com
- https://www.gnu.org/software/libc/



Image source



uClibc-ng

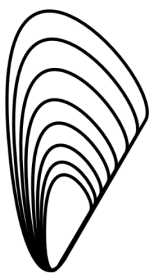
- https://uclibc-ng.org/
- A continuation of the old uClibc project, license: LGPL
- Lightweight C standard library for small embedded systems
  - High configurability: many features can be enabled or disabled through a menuconfig interface.
  - Supports most embedded architectures, including MMU-less ones (ARM Cortex-M, Blackfin, etc.). The only standard library supporting ARM noMMU.
  - No guaranteed binary compatibility. May need to recompile applications when the library configuration changes.
  - Some features may be implemented later than on glibc (real-time, floating-point operations...)
  - Focus on size (RAM and storage) rather than performance
  - Size on armv7hf, version 1.0.34: `libc`: 712 KB, source: https://toolchains.bootlin.com
- Actively supported, supported by Buildroot but not by Yocto Project.



musl C standard library

https://www.musl-libc.org/

- A lightweight, fast and simple standard library for embedded systems
- Created while uClibc's development was stalled
- In particular, great at making small static executables, which can run anywhere, even on a system built from another C standard library.
- More permissive license (MIT), making it easier to release static executables. We will talk about the requirements of the LGPL license (glibc, uClibc) later.
- Supported by build systems such as Buildroot and Yocto Project.
- Used by the Alpine Linux lightweight distribution (https://www.alpinelinux.org/)
- Size on armv7hf, version 1.2.0: `libc`: 748 KB, source: https://toolchains.bootlin.com





Other smaller C libraries

- Several other smaller C libraries exist, but they do not implement the full POSIX interface required by most Linux applications
- They can run only relatively simple programs, typically to make very small static executables and run in very small root filesystems.
- Therefore not commonly used in most embedded Linux systems
- Choices:
  - Newlib, https://sourceware.org/newlib/, maintained by Red Hat, used mostly in Cygwin, in bare metal and in small POSIX RTOS.
  - Klibc, https://en.wikipedia.org/wiki/Klibc, from the kernel community, designed to implement small executables for use in an *initramfs* at boot time.



Advice for choosing the C standard library

- Advice to start developing and debugging your applications with *glibc*, which is the most standard solution
- If you have size constraints, try to compile your app and then the entire filesystem with *uClibc* or *musl*
  - The size advantage of *uClibc* or *musl*, which used to be a significant argument, is less relevant with today's storage capacities.
  - Smaller binaries and filesystems remain useful when optimizing boot time, though, typically booting on a filesystem loaded in RAM, and to reduce the size of container and virtual machine images (one of the use cases of Alpine Linux).
- If you run into trouble, it could be because of missing features in the C standard library.
- In case you wish to make static executables, *musl* will be an easier choice in terms of licensing constraints.



Linux vs. bare-metal toolchain

- A **Linux toolchain**
  - is a toolchain that includes a Linux-ready C standard library, which uses the Linux system calls to implement system services
  - can be used to build Linux user-space applications, but also bare-metal code (firmware, bootloader, Linux kernel)
  - is identified by the `linux` OS identifier in the toolchain tuple: `arm-linux`, `arm-none-linux-gnueabihf`
- A **bare metal toolchain**
  - is a toolchain that does not include a C standard library, or a very minimal one that isn't tied to a particular operating system
  - can be used to build only bare-metal code (firmware, bootloader, Linux kernel)
  - is identified by the `none` OS identifier in the toolchain tuple: `arm-none-eabi`, `arm-none-none-eabi` (vendor is `none`, OS is `none`)



An alternate compiler suite: LLVM

- Most Embedded Linux projects use toolchains based on the GNU project: GCC compiler, binutils, GDB debugger
- The LLVM project has been developing an alternative compiler suite:
  - Clang, C/C++ compiler, https://clang.llvm.org/
  - LLDB, debugger, https://lldb.llvm.org/
  - LLD, linker, https://lld.llvm.org/
  - and more, see https://llvm.org/
- While they are used by several high-profile projects, they are not yet in widespread use in most Embedded Linux projects.
- Initially had better code optimization and diagnostics than GCC, but thanks to having competition, GCC has improved significantly in this area.
- Available under MIT/BSD licenses
- https://en.wikipedia.org/wiki/LLVM



Toolchain Options



ABI

- When building a toolchain, the ABI used to generate binaries needs to be defined
- ABI, for *Application Binary Interface*, defines the calling conventions (how function arguments are passed, how the return value is passed, how system calls are made) and the organization of structures (alignment, etc.)
- All binaries in a system are typically compiled with the same ABI, and the kernel must understand this ABI.
- On ARM 32-bit, two main ABIs: *EABI* and *EABIhf*
  - *EABIhf* passes floating-point arguments in floating-point registers → needs an ARM processor with a FPU
- On RISC-V, several ABIs: *ilp32*, *ilp32f*, *ilp32d*, *lp64*, *lp64f*, and *lp64d*
- https://en.wikipedia.org/wiki/Application_Binary_Interface



Floating point support

- All ARMv7-A (32-bit) and ARMv8-A (64-bit) processors have a floating point unit
- RISC-V cores with the F extension have a floating point unit
- Some older ARM cores (ARMv4/ARMv5) or some RISC-V cores may not have a floating point unit
- For processors without a floating point unit, two solutions for floating point computation:
  - Generate *hard float code* and rely on the kernel to emulate the floating point instructions. This is very slow.
  - Generate *soft float code*, so that instead of generating floating point instructions, calls to a user space library are generated
- Decision taken at toolchain configuration time
- For processors with a floating point unit, sometimes different FPU are possible. For example on ARM: VFPv3, VFPv3-D16, VFPv4, VFPv4-D16, etc.



CPU optimization flags

- GNU tools (gcc, binutils) can only be compiled for a specific target architecture at a time (ARM, x86, RISC-V...)
- gcc offers further options:
  - `-march` allows to select a specific target instruction set
  - `-mtune` allows to optimize code for a specific CPU
  - `-mcpu` can be used instead, it allows gcc to infer both the the target instruction set and CPU optimizations, eg:
    - `-mcpu=cortex-a8` implies `-march=armv7` and `-mtune=cortex-a8`
  - https://gcc.gnu.org/onlinedocs/gcc/ARM-Options.html
- At the GNU toolchain compilation time, values can be chosen. They are used:
  - As the default values for the cross-compiling tools, when no other `-march`, `-mtune`, `-mcpu` options are passed
  - To compile the C library
- Note: LLVM (Clang, LLD...) utilities support multiple target architectures at once.



Obtaining a Toolchain



Building a toolchain manually

- Building a cross-compiling toolchain manually is a fairly difficult process
- Lots of details to learn: many components to build with complicated configuration
- Typical process is:
  - Build dependencies of binutils/gcc (GMP, MPFR, ISL, etc.)
  - Build *binutils*
  - Build a baremetal, first stage *GCC*
  - Extract kernel headers from the Linux source code
  - Build the C library using the first stage GCC
  - Build the second stage and final *GCC* supporting the Linux OS and the C library.
- Many decisions to make about the components: C library, gcc and binutils versions, ABI, floating point mechanisms, etc. Not trivial to find correct combinations of these possibilities
- See the Crosstool-NG documentation for details on how toolchains are built.
- Talk: *Anatomy of Cross-Compilation Toolchains*, by Thomas Petazzoni, ELCE 2017, video and slides



Get a pre-compiled toolchain

- Solution that many people choose
  - Advantage: it is the simplest and most convenient solution
  - Drawback: you can't fine tune the toolchain to your needs
- Make sure the toolchain you find meets your requirements: CPU, endianness, C library, component versions, version of the kernel headers, ABI, soft float or hard float, etc.
- Some possibilities:
  - Toolchains packaged by your distribution, for example Ubuntu package gcc-arm-linux-gnueabihf or Fedora gcc-arm-linux-gnu. Often limited to ARM/ARM64 with glibc.
  - Bootlin's GNU toolchains, most CPU architectures, with glibc/uClibc/musl: https://toolchains.bootlin.com
  - ARM and ARM64 toolchains released by ARM



Example of toolchains from ARM: downloading

x86_64 Linux hosted cross compilers

AArch32 bare-metal target (arm-none-eabi)

- gcc-arm-10.3-2021.07-x86_64-arm-none-eabi.tar.xz
- gcc-arm-10.3-2021.07-x86_64-arm-none-eabi.tar.xz.asc

AArch32 target with hard float (arm-none-linux-gnueabihf)

- gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz
- gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz.asc

AArch64 ELF bare-metal target (aarch64-none-elf)

- gcc-arm-10.3-2021.07-x86_64-aarch64-none-elf.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64-none-elf.tar.xz.asc

AArch64 GNU/Linux target (aarch64-none-linux-gnu)

- gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz.asc

AArch64 GNU/Linux target (aarch64_be-none-linux-gnu)

- gcc-arm-10.3-2021.07-x86_64-aarch64_be-none-linux-gnu.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64_be-none-linux-gnu.tar.xz.asc

From Arm GNU Toolchains



Example of toolchains from ARM: using

```
$ wget https://developer.arm.com/-/media/Files/downloads/gnu-a/10.3-2021.07/binrel/[...]
    [...]gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz

$ tar xf gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz

$ cd gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/

$ ./bin/arm-none-linux-gnueabihf-gcc -o test test.c

$ file test test: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked,
interpreter /lib/ld-linux-armhf.so.3, [...]
   for GNU/Linux 3.2.0, with debug_info, not stripped
```



Toolchain building utilities

Another solution is to use utilities that **automate the process of building the toolchain**

- Same advantage as the pre-compiled toolchains: you don't need to mess up with all the details of the build process
- But also offers more flexibility in terms of toolchain configuration, component version selection, etc.
- Allows to rebuild the toolchain if needed to fix a bug or security issue.
- They also usually contain several patches that fix known issues with the different components on some architectures
- Multiple tools with identical principle: shell scripts or Makefile that automatically fetch, extract, configure, compile and install the different components

Toolchain building utilities (2)

Crosstool-ng

- Rewrite of the older Crosstool, with a menuconfig-like configuration system
- Feature-full: supports uClibc, glibc and musl, hard and soft float, many architectures
- Actively maintained
- https://crosstool-ng.github.io/



Crosstool-NG is a versatile (cross) toolchain generator. It supports many architectures and components and has a simple yet powerful menuconfig-style interface. Please read the introduction and refer to the documentation for more information.

See what the users of crosstool-NG have to say!

Latest sources, bugs, questions? Head over to Crosstool-NG at GitHub!

News

May 7, 2022

Released 1.25.0

Get the 1.25.0 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .

continued ...

Apr 22, 2022

Released 1.25.0_rc2

Get the 1.25.0_rc2 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .

Mar 24, 2022

Released 1.25.0_rc1

Get the 1.25.0_rc1 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .



Toolchain building utilities (3)

Many root filesystem build systems also allow the construction of a cross-compiling toolchain

- **Buildroot**
  - Makefile-based. Can build glibc, uClibc and musl based toolchains, for a wide range of architectures. Use `make sdk` to only generate a toolchain.
  - https://buildroot.org
- **PTXdist**
  - Makefile-based, maintained mainly by *Pengutronix*, supporting only glibc and uClibc (version 2023.01 status)
  - https://www.ptxdist.org/
- **OpenEmbedded / Yocto Project**
  - A featureful, but more complicated build system, supporting only glibc and musl.
  - https://www.openembedded.org/
  - https://www.yoctoproject.org/



Crosstool-NG: download

- Getting Crosstool-NG

```
$ git clone https://github.com/crosstool-ng/crosstool-ng.git
```

- Using a well-known stable version

```
$ cd crosstool-ng
$ git checkout crosstool-ng-1.28.0
```

- As we're fetching from Git, the `configure` script needs to be generated:

```
$ ./bootstrap
```



Crosstool-NG: installation

- Installation can be done:
  - system-wide, for example in `/usr/local`, the `ct-ng` command is then available globally

```
$ ./configure
$ make
$ sudo make install
```

  - or just locally in the source directory, the `ct-ng` command will be invoked from this directory

```
$ ./configure --enable-local
$ make
```

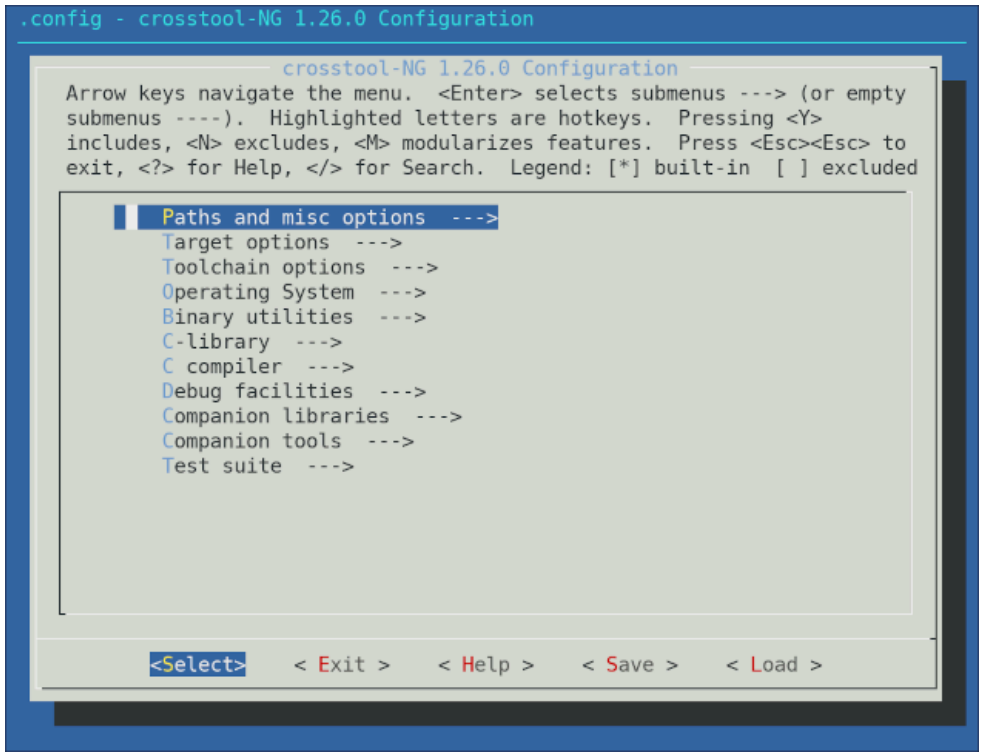
- In our labs, we will use the second method
- Note: the `make` invocation doesn't build any toolchain, it builds the `ct-ng` executable.



Crosstool-NG: toolchain configuration

- Once installed, the `ct-ng` tool allows to configure and build an arbitrary number of toolchains
- Its configuration system is based on *kconfig*, like the Linux kernel configuration system
- Configuration of the toolchain to build stored in a `.config` file
- Example configurations provided with Crosstool-NG
  - List: `./ct-ng list-samples`
  - Load an example: `./ct-ng <sample-name>`, replaces `.config`
  - For example `./ct-ng aarch64-unknown-linux-gnu`
  - No sample loaded → default Crosstool-NG configuration is a bare-metal toolchain for the *Alpha* CPU architecture!
- The configuration can then be refined using either:
  - `./ct-ng menuconfig`
  - `./ct-ng nconfig`

Crosstool-NG: toolchain configuration



./ct-ng menuconfig



Crosstool-NG: toolchain building

- To build the toolchain

```
./ct-ng build
```

  This will automatically download all the needed dependencies, and build all toolchain components in the right order, with the specified configuration.
- By default the results go in `$HOME/x-tools/`, as defined by the option in *Paths and misc options*



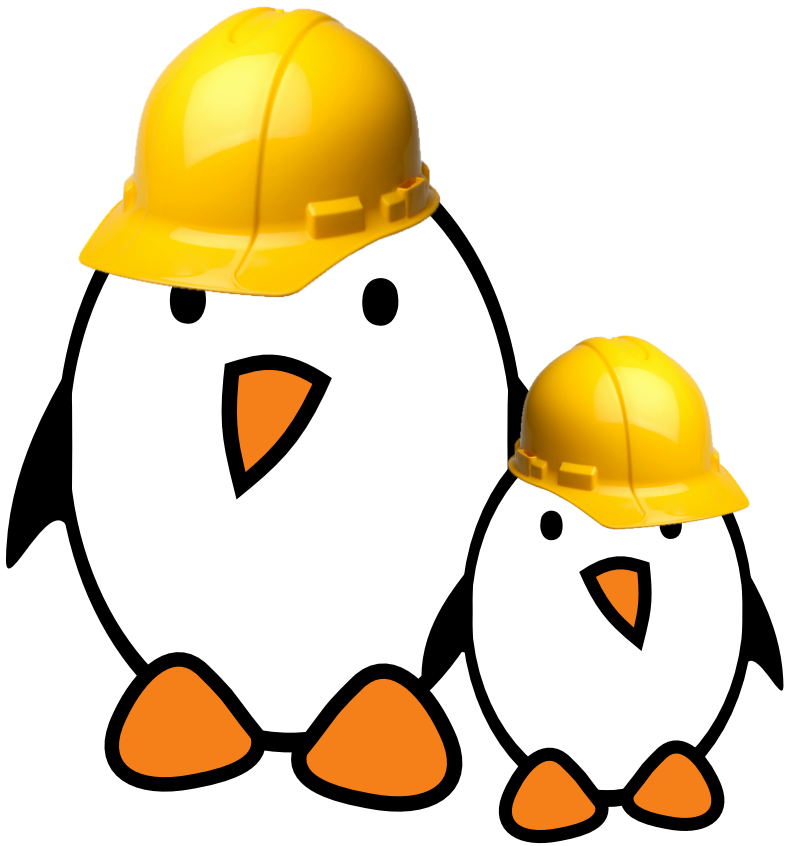
Important toolchain contents

- `bin/`: cross compilation tool binaries
  - This directory can be added to your `PATH` to ease usage of the toolchain
  - Sometimes with symlinks for shorter names

```
arm-linux-gcc -> arm-cortexa7-linux-uclibcgnueabihf-gcc
```

- `<arch-tuple>/sysroot`: *sysroot* directory
  - `<arch-tuple>/sysroot/lib`: C library, GCC runtime, C++ standard library compiled for the target
  - `<arch-tuple>/sysroot/usr/include`: C library headers and kernel headers

Practical lab - Using Crosstool-NG



Time to build your toolchain

- Getting and configuring Crosstool-NG
- Executing it to build a custom cross-compilation toolchain
- Exploring the contents of the toolchain